UNIT – 2

ASSESSMENT – 3

REPORT

1. Title: *A* Search Algorithm*

Objective:

To implement the A* Search Algorithm to determine the shortest path between the source node and the goal node using actual path cost and heuristic values.

Algorithm Used:

- A* Search Algorithm

Software Requirements:

- Python 3.x
- Visual Studio Code / PyCharm / Google Colab

Hardware Requirements:

- Computer with at least 4 GB RAM

Methodology:

1. Create a weighted graph.
2. Assign heuristic values to each node.
3. Read the source and goal nodes.
4. Calculate f(n) = g(n) + h(n).
5. Expand the node with the lowest f(n) value.

6. Update the Open and Closed Lists.
7. Repeat until the goal node is reached.
8. Display the optimal path and total cost.

Expected Output:

- Open List
- Closed List
- Optimal Path
- Total Cost

Advantages:

- Finds the shortest path.
- Uses heuristic information.
- Reduces unnecessary search.
- Guarantees an optimal solution with an admissible heuristic.

Applications:

- GPS Navigation
- Robot Navigation
- Network Routing
- Game AI
- Path Planning **Result:**

The A* Search Algorithm was successfully implemented and the optimal path between the source and goal nodes was obtained.

2.Title: Minimax with Alpha-Beta Pruning

Objective:

To implement the Minimax Algorithm with Alpha-Beta Pruning to determine the optimal move while reducing the number of nodes evaluated.

Algorithm Used:

- **Minimax Algorithm**
- **Alpha-Beta Pruning**

Software Requirements:

- Python 3.x
- Visual Studio Code / PyCharm / Google Colab

Hardware Requirements:

- Computer with at least 4 GB RAM

Methodology:

1. Construct the game tree.
2. Assign values to all leaf nodes.
3. Start evaluation from the root node.

4. Alternate between MAX and MIN levels.
5. Update Alpha (α) and Beta (β) values.
6. Prune branches whenever $\beta \leq \alpha$.
7. Determine the optimal move.
8. Display the final Minimax value.

Expected Output:

- Alpha (α) values
- Beta (β) values
- Pruned nodes
- Best move
- Final Minimax value

Advantages:

- Reduces computation time.
- Eliminates unnecessary node evaluation.
- Produces the optimal move.
- Improves search efficiency.

Applications:

- Chess
- Tic-Tac-Toe
- Checkers
- Real-Time Strategy Games

- Game AI

Result:

The Minimax Algorithm with Alpha-Beta Pruning was successfully implemented, and the optimal move was determined while reducing the number of nodes evaluated through pruning.